exp

March 16, 2026

[1]:
```python
import numpy as np
from optic.models.devices import mzm, photodiode
from optic.models.channels import linearFiberChannel
from optic.comm.sources import bitSource
from optic.comm.modulation import modulateGray
from optic.comm.metrics import bert
from optic.dsp.core import firFilter, pulseShape, upsample, pnorm, anorm
from optic.utils import parameters, dBm2W
from scipy.special import erfc

import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Model
```

[2]:
```python
# simulation parameters
SpS = 16  # samples per symbol
M = 2  # order of the modulation format
Rs = 10e9  # Symbol rate
Fs = SpS * Rs  # Signal sampling frequency (samples/second)
Pi_dBm = 3  # laser optical power at the input of the MZM in dBm
Pi = dBm2W(Pi_dBm)  # convert from dBm to W

# Bit source parameters
paramBits = parameters()
paramBits.nBits = 2**18  # number of bits to be generated
paramBits.mode = 'random' # mode of the bit source
paramBits.seed = 123      # seed for the random number generator

# pulse shaping parameters
paramPulse = parameters()
paramPulse.pulseType = 'nrz'  # pulse shape type
paramPulse.SpS = SpS     # samples per symbol

# MZM parameters
paramMZM = parameters()
paramMZM.Vpi = 2
paramMZM.Vb = -paramMZM.Vpi / 2
```

```
# linear fiber optical channel parameters
paramCh = parameters()
paramCh.L = 100         # total link distance [km]
paramCh.alpha = 0.2     # fiber loss parameter [dB/km]
paramCh.D = 16          # fiber dispersion parameter [ps/nm/km]
paramCh.Fc = 193.1e12   # central optical frequency [Hz]
paramCh.Fs = Fs

# photodiode parameters
paramPD = parameters()
paramPD.ideal = False
paramPD.B = Rs
paramPD.Fs = Fs
paramPD.seed = 456  # seed for the random number generator


def optical_chain(symbTx):
    # upsampling
    symbolsUp = upsample(symbTx, SpS)

    # pulse shaping
    pulse = pulseShape(paramPulse)
    sigTx = firFilter(pulse, symbolsUp)
    sigTx = anorm(sigTx) # normalize to 1 Vpp


    # artifical distortion (non linear)
    alpha = 0.5
    sigTx = sigTx + alpha * sigTx**3


    # optical modulation
    Ai = np.sqrt(Pi)  # ideal cw laser constant envelope
    sigTxo = mzm(Ai, sigTx, paramMZM)

    # linear fiber channel model
    sigCh = linearFiberChannel(sigTxo, paramCh)

    # noisy PD (thermal noise + shot noise + bandwidth limit)
    I_Rx = photodiode(sigCh, paramPD)

    # capture samples in the middle of signaling intervals
    I_Rx = I_Rx[0::SpS]

    return I_Rx
```

[3]:
```
bitsTx = bitSource(paramBits)
symbTx = modulateGray(bitsTx, M, "pam")
I_Rx = optical_chain(symbTx)

BER, Q = bert(I_Rx, bitsTx)

print(f"{BER:<10.2e}") # Perf w/out DPD
```

```
3.31e-03
```

0.1 Model Arch & Training

[4]:
```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, initializers, Sequential
from optic.models.devices import mzm, photodiode
from optic.models.channels import linearFiberChannel
from optic.comm.sources import bitSource
from optic.comm.modulation import modulateGray
from optic.comm.metrics import bert
from optic.dsp.core import firFilter, pulseShape, upsample, anorm
from optic.utils import parameters, dBm2W


best_ber = float('inf')

def build_model():
    inputs = layers.Input(shape=(None, 1))

    kernel_size = 20
    impulse_init = np.zeros((kernel_size, 1, 1))
    impulse_init[kernel_size // 2, 0, 0] = 1.0

    sec_a = layers.Conv1D(1, kernel_size, padding='same',
                          kernel_initializer=initializers.
 ↪Constant(impulse_init),
                          bias_initializer='zeros')(inputs)

    # sec_a = inputs

    # sec_a = layers.Conv1D(1, kernel_size, padding='same')(inputs)

    x = layers.Dense(20, activation=layers.LeakyReLU(0.1))(sec_a)
    x = layers.Dense(20, activation=layers.LeakyReLU(0.1))(x)
    x = layers.Dense(1, activation=layers.LeakyReLU(0.1))(x)
    nonlinear_out = layers.Dense(1, activation='linear')(x)
```

```
    outputs = nonlinear_out

    return Model(inputs, outputs)



dpd_model = build_model()
dpd_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-2),␣
 ↪loss='mse')



# Step A: Data Generation
bitsTx = bitSource(paramBits)
symbTx = modulateGray(bitsTx, M, "pam")
x_input = symbTx.reshape(-1, 8192, 1)

# DPD training loop
for iteration in range(30):
    z_dpd = dpd_model.predict(x_input, verbose=0) if iteration != 0 else x_input
    z_signal = z_dpd.flatten()

    I_Rx = optical_chain(z_signal)

    # Performance Monitoring & checkpointing (before fit, so saved weights␣
 ↪match reported BER)
    if iteration==0:
        print("Iteration | Q-factor  | BER")
    BER, Q = bert(I_Rx, bitsTx)
    print(f"{iteration+1:<5} | {Q:<10.2f} | {BER:<10.2e}")

    if BER < best_ber:
        dpd_model.save_weights('best_weights.weights.h5')
        best_ber = BER

    # Normalize received signal for training [this works but why? if u remove␣
 ↪it model diverges]. => see logs feb 26th
    y_received = (I_Rx - np.mean(I_Rx)) / np.std(I_Rx)
    y_received = y_received.reshape(-1, 8192, 1)

    dpd_model.fit(y_received, z_dpd, epochs=100, verbose=0)

print("\nTraining Complete.")
```

```
2026-03-09 12:44:30.102745: I metal_plugin/src/device/metal_device.cc:1154]
Metal device set to: Apple M3
2026-03-09 12:44:30.102772: I metal_plugin/src/device/metal_device.cc:296]
```

```
systemMemory: 24.00 GB
2026-03-09 12:44:30.102776: I metal_plugin/src/device/metal_device.cc:313]
maxCacheSize: 8.88 GB
2026-03-09 12:44:30.102791: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305]
Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel
may not have been built with NUMA support.
2026-03-09 12:44:30.102799: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271]
Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0
MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id:
<undefined>)

Iteration | Q-factor  | BER
1     | 2.59       | 3.31e-03

2026-03-09 12:44:31.084556: I
tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117]
Plugin optimizer for device_type GPU is enabled.

2     | 0.76       | 3.25e-01
3     | 1.61       | 6.27e-02
4     | 3.30       | 3.28e-04
5     | 3.56       | 1.45e-04
6     | 2.96       | 1.40e-03
7     | 2.94       | 1.53e-03
8     | 2.89       | 1.84e-03
9     | 2.83       | 2.13e-03
10    | 2.81       | 2.31e-03
11    | 2.79       | 2.43e-03
12    | 2.78       | 2.47e-03
13    | 2.77       | 2.52e-03
14    | 2.75       | 2.60e-03
15    | 2.73       | 2.69e-03
16    | 2.72       | 2.76e-03
17    | 2.71       | 2.80e-03
18    | 2.70       | 2.80e-03
19    | 2.70       | 2.86e-03
20    | 2.69       | 2.90e-03
21    | 2.68       | 2.94e-03
22    | 2.67       | 2.98e-03
23    | 2.66       | 3.01e-03
24    | 2.64       | 3.09e-03
25    | 2.63       | 3.12e-03
26    | 2.62       | 3.14e-03
27    | 2.65       | 3.08e-03
28    | 2.64       | 3.11e-03
29    | 2.64       | 3.12e-03
30    | 2.64       | 3.12e-03
```

```
Training Complete.
```

[5]:
```python
def optical_chain_w_dpd(symbTx, DPD_ON=False):
    x_input = symbTx.reshape(-1, 8192, 1)

    if DPD_ON:
        z_dpd = dpd_model.predict(x_input, verbose=0)
        z_signal = z_dpd.flatten()
    else:
        z_signal = symbTx

    I_Rx = optical_chain(z_signal)

    return I_Rx
```

[6]:
```python
paramBits.seed = 456

bitsTx = bitSource(paramBits)
symbTx = modulateGray(bitsTx, M, "pam")


dpd_model.load_weights('best_weights.weights.h5')
I_Rx = optical_chain_w_dpd(symbTx, DPD_ON=True)

BER, Q = bert(I_Rx, bitsTx)
print(f"{Q:<10.2f} | {BER:<10.2e}")
```

```
3.56       | 1.41e-04
```